Lecture #1:

Recursive Function + Structure

Recursive function is a function that call itself.

```
int f(int x)
    {
        if(x==0)
          return 1;
        else
          return 2 * f(x-1);
    }
```

Problems defined recursively

- There are many problems whose solution can be defined recursively

Example *n factorial*

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n*(n-1)! & \text{if } n > 0 \end{cases}$$

(***recursive*** **solution)**

- **Recursive implementation**

```
int Factorial(int n)
{
      if (n==0)  // base case
        return 1;
      else
        return n * Factorial(n-1);
}
```

- Recursive implementation

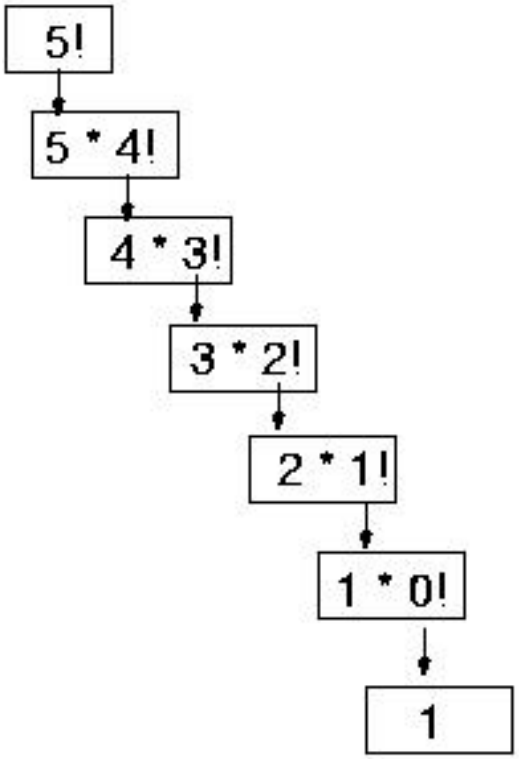
```
int Factorial(int n)
  {
    if (n==0)  // base case
     return 1;
    else
     return n * Factorial(n-1);
  }
```

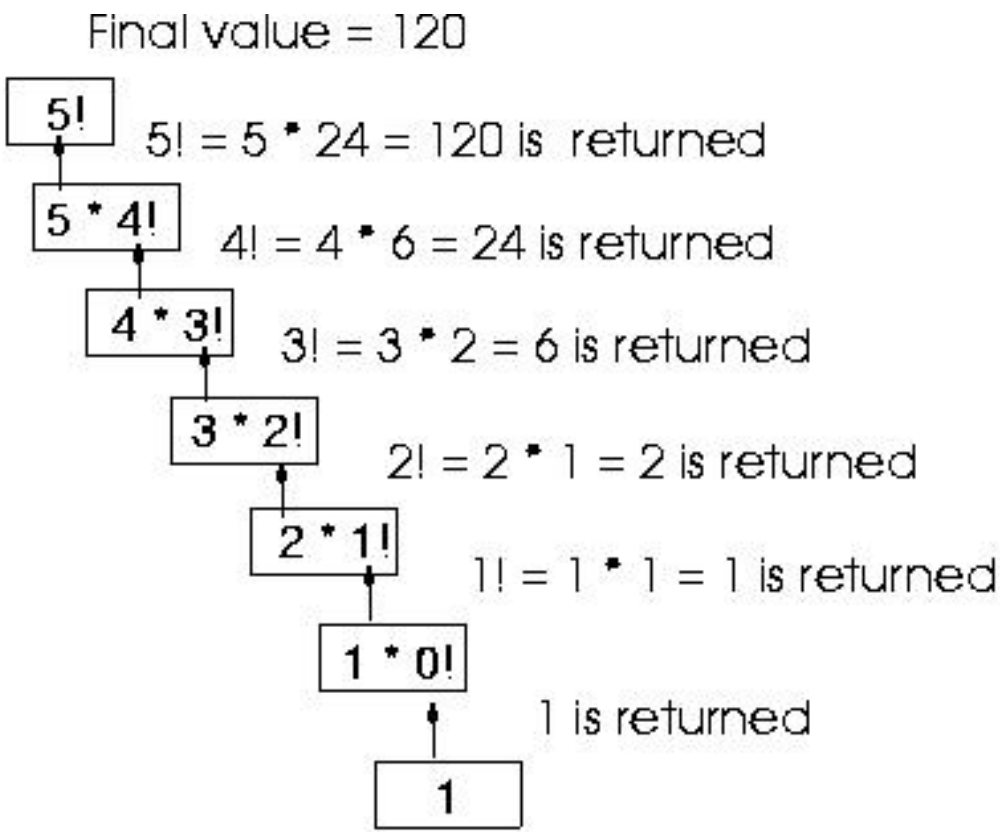
- Iterative implementation

```
int Factorial(int n)
{
      int fact = 1;

      for(int count = 2; count <= n; count++)
        fact = fact * count;

      return fact;
}
```





Recursion vs. iteration

- Iteration can be used in place of recursion
  - An iterative algorithm uses a *looping construct*
  - A recursive algorithm uses a *branching structure*
- Recursive solutions are often less efficient, in terms of both *time* and *space*, than iterative solutions
- Recursion can simplify the solution of a problem, often resulting in *shorter*, more easily understood source code

How do I write a recursive function?

- Determine the size factor
- Determine the base case(s)

  (the one for which you know the answer)
- Determine the general case(s)

  (the one where the problem is expressed as a smaller version of itself)
- Verify the algorithm

  (use the "Three-Question-Method")

- Recursive implementation

```
int Factorial(int n)
{
    if (n==0)  // base case
      return 1;
    else
      return n * Factorial(n-1);
}
```

Three-Question Verification Method

1. The Base-Case Question:

   Is there a nonrecursive way out of the function, and does the routine work correctly for this " base" case?

2. The Smaller-Caller Question:

   Does each recursive call to the function involve a smaller case of the original problem, leading inescapably to the base case?

3. The General-Case Question:

   Assuming that the recursive call(s) work correctly, does the whole function work correctly?

What happens when a function is called?

- An activation record is stored into a stack (run-time stack)

1) The computer has to stop executing function *b* and starts executing function a
2) Since it needs to come back to function *b* later, it needs to store everything about function b that is going to need (x, y, z and the place to start executing upon return)
3) Then, x from b is bounded to w from a
4) Control is transferred to function a

- After function a is executed, the activation record is popped out of the run-time stack
- All the old values of the parameters and variables in function b are restored and the return value of function a replasced a(x) in the assignment statement

```
int a(int w)
{
    return w + w;
}


int b(int x, int y)
{
    int z;
    z = a(x) + y;
    return z;
}
```

What happens when a recursive function is called?

- Except the fact that the calling and called functions have the same name, there is really no difference between recursive and nonrecursive calls

```
int f(int x)
  {
      if(x==0)
        return 1;
      else
        return 2 * f(x-1);
  }
```

Structures

- *Structure is a data structure* used to process data of multiple data types.

- *Structure example:*

  Student Record:
  - Name a string
  - HW Grades an array of 3 doubles
  - Test Grades an array of 2 doubles
  - Final Average a double

Structure Members

- Each *thing* in a structure is called *member.*
- Each *member* has a name, a type and a value.
- Names follow the rules for variable names.
- Types can be any defined type.

Example Structure Definition

```
struct StudentRecord {
   char *name;     // student name
   double hw[3];   // homework grades
   double test[2]; // test grades
   double ave;     // final average
};
```

Using a struct

- By defining a structure you create a new data type.
- Once a struct is defined, you can create variables of the new type.

```
StudentRecord stu;
```

Accessing Members

- You can treat the members of a struct just like variables.
- You need to use the *member access operator* '.' (pronounced "dot"):

```
cout << stu.name << endl;
stu.hw[2] = 82.3;
stu.ave = total/100;
```

Structure Assignment

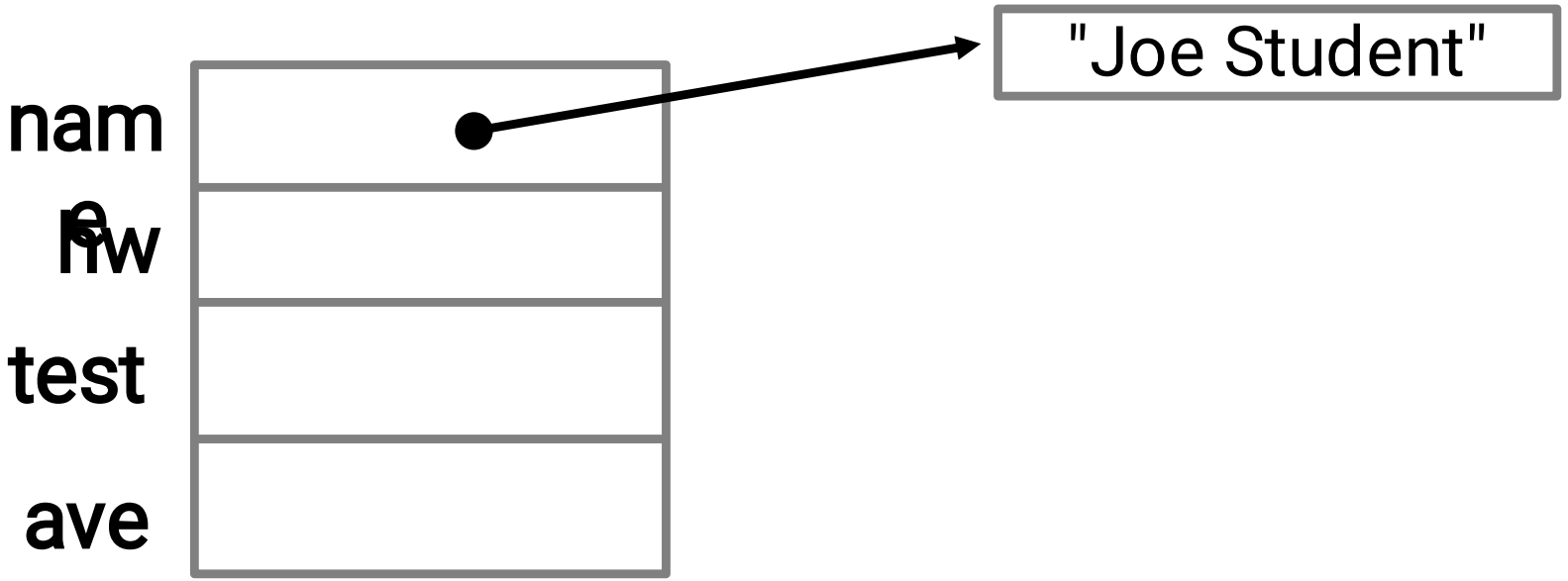
- You can use structures just like variables:

```
StudentRecord s1,s2;
s1.name = "Joe Student";
…
s2 = s1;
```

Copies the entire structure

Be Careful

- If a member is a pointer, *copying* means copying the pointer (not what is pointed to).



Pointers to Structures

- Pointers to structures are used often.
- There is another *member access operator* used with pointers: **->**

it looks like a "pointer"!

```
StudentRecord *sptr;
...
cout << "Name is" << sptr->name;
cout << "Ave is " << sptr->ave;
```

Sample Function

```
void update_average( StudentRecord *stu)
{
   double tot=0;
   for (int i=0;i<3;i++)
      tot += stu->hw[i];
   for (int i=0;i<3;i++)
      tot += stu->test[i];
   stu->ave = tot/5;
}
```

Structures - Example

```
#include <iostream>
using namespace std;
struct student
{
    char name[30];
    float marks;
}   student1, student2;

int main()
{
    student student3;
    char s1[30], s2[30], s3[30];
    float f1,f2,f3;
    cout <<" Name and Mark of student1:   \n"; cin>>s1; cin>>f1;
    cout <<" Name and Mark of student2:   \n"; cin>>s2; cin>>f2;
    cout <<" Name and Mark of student3:   \n"; cin>>s3; cin>>f3;
    strcpy(student1.name, s1);   strcpy(student2.name, s2);  strcpy(student3.name, s3);
    student1.marks = f1;    student2.marks = f2;   student3.marks = f3;
    cout <<" Name and Mark of student1: \n" << student1.name << "  "<<student1.marks;
    cout <<"\n Name and Mark of student2:  \n" << student2.name << "  "<<student2.marks;
    cout <<"\n Name and Mark of student3: \n" << student3.name << "  "<<student3.marks;
    return 0;
}
```